ACP SI AF

1. FUNCTII

```
import numpy as np
import pandas as pd
from pandas.api.types import is_numeric_dtype


def nan_replace(t):
    assert isinstance(t, pd.DataFrame)
    for c in t.columns:
        if t[c].isna().any():
            if is_numeric_dtype(t[c]):
                t[c].fillna(t[c].mean(), inplace=True)
            else:
                t[c].fillna(t[c].mode()[0], inplace=True)


class acp():
    def __init__(self, t, variabile_observate):
        assert isinstance(t, pd.DataFrame)
        self.__x = t[variabile_observate].values
        self.variabile_observate = variabile_observate

    @property
    def x(self):
        return self.__x

    def fit(self, std=True, nlib=0, procent_minimal_varianta=80):
        n, m = self.__x.shape
        # standardizarea datelor
        x_ = self.__x - np.mean(self.__x, axis=0)
        if std:
            x_ = x_ / np.std(self.__x, axis=0)
        # matricea de covarianta
        r_v = (1 / (n - nlib)) * x_.T @ x_
        valp, vecp = np.linalg.eig(r_v)  # sortate in ordine
descrescatoare
        k = np.flip(np.argsort(valp))
        self.__alpha = valp[k]
        self.__a = vecp[:, k]
        self.__c = x_ @ self.__a  # scoruri componente
        # @ produs matriceal
        procent_cumulat_varianta = np.cumsum(self.__alpha) * 100 /
sum(self.__alpha)
        k1 = np.where(procent_cumulat_varianta >
procent_minimal_varianta)[0][
                 0] + 1  # nr minim de componente pt a explica
prc_min_var
        if std:
            k2 = np.where(self.__alpha <= 1)[0][0]
        else:
            k2 = np.NAN
        eps = self.__alpha[1:] - self.__alpha[:m - 1]
        sigma = eps[1:] - eps[:m - 2]
        exista_negative = sigma < 0
        if any(exista_negative):
```

```
            k3 = np.where(exista_negative)[0][0] + 2
        else:
            k3 = np.NAN
        self.__k = (k1, k2, k3)
        if std:
            self.__r = self.__a * np.sqrt(self.__alpha)
        else:
            self.__r = np.corrcoef(x_, self.__c, rowvar=False)[:m, m:]

    @property
    def r(self):
        return self.__r

    @property
    def k(self):
        return self.__k

    @property
    def alpha(self):
        return self.__alpha

    @property
    def a(self):
        return self.__a

    @property
    def c(self):
        return self.__c

    def tabelare_varianta(self):
        varianta_cumulata = np.cumsum(self.__alpha)
        procent_varianta = self.__alpha * 100 / sum(self.__alpha)
        procent_cumulat = np.cumsum(procent_varianta)
        return pd.DataFrame({
            "Varianta": self.__alpha,
            "Varianta cumulata": varianta_cumulata,
            "Procent varianta": procent_varianta,
            "Procent cumulat": procent_cumulat
        }, ["C" + str(i) for i in range(1, len(self.__alpha) + 1)])


def calcul_criterii(alpha, procent_minimal_varianta=70):
    m = len(alpha)
    procent_cumulat_varianta = np.cumsum(alpha) * 100 / m
    k1 = np.where(procent_cumulat_varianta >
procent_minimal_varianta)[0][0] + 1
    k2 = np.where(alpha <= 1)[0][0]
    eps = alpha[1:] - alpha[:m - 1]
    sigma = eps[1:] - eps[:m - 2]
    exista_negative = sigma < 0
    if any(exista_negative):
        k3 = np.where(exista_negative)[0][0] + 2
    else:
        k3 = np.NAN
    return (k1, k2, k3)

# eps: Diferențele dintre valorile proprii consecutive.
# sigma: Diferențele dintre diferențele valorilor proprii consecutive.
```

2. GRAFICE

```
import matplotlib.pyplot as plt
import numpy as np
from seaborn import heatmap


def plot_varianta(alpha, k, procent_varianta=80,
eticheta_x="Componenta"):
    fig = plt.figure(figsize=(10, 7))
    ax = fig.add_subplot(1, 1, 1)
    assert isinstance(ax, plt.Axes)
    ax.set_title("Plot varianta")
    ax.set_xlabel(eticheta_x)
    ax.set_ylabel("Varianta")
    x = np.arange(1, len(alpha) + 1)
    ax.set_xticks(x)
    ax.plot(x, alpha) # val de pe axa x sunt date de x, de pe y sunt
date de alpha
    ax.scatter(x, alpha, c='r')
    ax.axvline(k[0], c='g', label='Varianta minimala' +
str(procent_varianta) + '%')
    if not np.isnan(k[1]):
        ax.axvline(k[1], c='c', label='Kaiser')
    if not np.isnan(k[2]):
        ax.axvline(k[2], c='m', label='Cattell')
    ax.legend()
    plt.savefig('Plot_varianta')


def corelograma(t, vmin=-1, titlu='Corelatii factoriale',
cmap='coolwarm'):
    fig = plt.figure(titlu, figsize=(10, 7))
    ax = fig.add_subplot(1, 1, 1)
    assert isinstance(ax, plt.Axes)
    ax.set_title(titlu)
    heatmap(t, vmin=vmin, cmap=cmap, annot=True, ax=ax)
    plt.savefig(titlu)


def show():
    plt.show()


def scatter(t, var1='C1', var2='C2', titlu='Plot scoruri',
corelatii=False):
    fig = plt.figure(titlu + ',' + var1 + ',' + var2, figsize=(10, 7))
    ax = fig.add_subplot(1, 1, 1, aspect=1)
    assert isinstance(ax, plt.Axes)
    ax.set_title(titlu)
    ax.set_xlabel(var1)
    ax.set_ylabel(var2)
    ax.axvline(0)
    ax.axhline(0)
    if corelatii:
        theta = np.arange(0, np.pi * 2, 0.01) # Se creează un array
theta care conține valori de la 0 la 2π (echivalentul a 360 de grade) cu
un pas de 0.01. Aceasta reprezintă un set de unghiuri care acoperă
întregul cerc
        ax.plot(np.cos(theta), np.sin(theta), c='m')
        ax.plot(0.7 * np.cos(theta), 0.7 * np.sin(theta), c='g')
```

```
    ax.scatter(t[var1], t[var2], c='r')
    for i in range(len(t)):
        ax.text(t[var1].iloc[i], t[var2].iloc[i], t.index[i])
```

3. MAIN ACP

```
import pandas as pd

from functii import *
from grafice import *

set_date = pd.read_csv('mortalitate_ro.csv', index_col=1)
variabile_observate = list(set_date)[1:]

valori_lipsa = set_date.isna().any().any()
if valori_lipsa:
    nan_replace(set_date)

m_acp = acp(set_date, variabile_observate)
m_acp.fit()

# AICI TABEL VARIANTA
tabel_varianta = m_acp.tabelare_varianta()

# CRITERII DE SELECTIE
print(m_acp.k)
plot_varianta(m_acp.alpha, m_acp.k)

# Analiza corelatiilor factoriale
r = m_acp.r
alpha = m_acp.alpha
etichete_componente = ['C' + str(i) for i in range(1, len(alpha) + 1)]
t_r = pd.DataFrame(r, variabile_observate, etichete_componente)
t_r.to_csv('R.csv')
corelograma(t_r)
scatter(t_r, titlu='Corelatii factoriale', corelatii=True)

# Analiza scorurilor
c = m_acp.c
s_c = c / np.sqrt(alpha)
t_s = pd.DataFrame(s_c, set_date.index, etichete_componente)
t_s.to_csv('Scoruri.csv')
scatter(t_s)
scatter(t_s, 'C3', 'C4')

# Cosinusuri
c2 = c * c
cos = (c2.T / np.sum(c2, axis=1)).T
t_cos = pd.DataFrame(cos, set_date.index, etichete_componente)
t_cos.to_csv('Cosin.csv')
cos_max = t_cos.apply(func=lambda x: x.index[x.argmax()], axis=1)
cos_max.name = "Componenta dominanta"
cos_max.to_csv('Cosin_max.csv')

# Contributii
contrib = c2 * 100 / np.sum(c2, axis=0)
t_contrib = pd.DataFrame(contrib, set_date.index, etichete_componente)
t_contrib.to_csv('Contrib.csv')
contrib_max = t_contrib.apply(func=lambda x: x.index[x.argmax()],
axis=0)
```

```
contrib_max.name = "Instanta dominanta"
contrib_max.to_csv("Contrib_max.csv")

# Comunalitati
r2 = r * r
comm = np.cumsum(r2, axis=1)
t_comm = pd.DataFrame(comm, variabile_observate, etichete_componente)
t_comm.to_csv('Comunalitati.csv')
corelograma(t_comm, vmin=0, titlu='Comunalitati', cmap='Greens')

show()
```

4. MAIN AF

```
import numpy as np
import pandas as pd
from functii import *
from grafice import *
from factor_analyzer import FactorAnalyzer, calculate_kmo,
calculate_bartlett_sphericity

date = pd.read_csv("mortalitate_ro.csv", index_col=1)
variabile_observate = list(date)[1:]

valori_lipsa = date.isna().any().any()
if valori_lipsa:
    nan_replace(date)

x = date[variabile_observate].values

# Analiza factorabilitate
test_bartlett = calculate_bartlett_sphericity(x)
print('Test Bartlett:', test_bartlett)
if test_bartlett[1] > 0.001:
    print('Nu exista factori comuni')
    exit(0)

index_kmo = calculate_kmo(x)
t_kmo = pd.DataFrame({
    "Index KMO": np.append(index_kmo[0], index_kmo[1])
}, variabile_observate + ['Total'])
t_kmo.to_csv('KMO.csv')
corelograma(t_kmo, 0, 'Index KMO', 'Blues')

# Construire model
n, m = x.shape
model_af = FactorAnalyzer(n_factors=m, rotation='varimax')
model_af.fit(x)

# Analiza variantei factoriale
varianta = model_af.get_factor_variance()
alpha = varianta[0]
etichete_factori = ['F' + str(i) for i in range(1, m + 1)]
tabel_varianta = pd.DataFrame({
    "Varianta": varianta[0],
    "Procent varianta": varianta[1] * 100,
    "Procent cumulat": varianta[2] * 100
}, etichete_factori)
tabel_varianta.to_csv('Varianta_af.csv')
criterii = calcul_criterii(alpha)
```

```
print("Criterii relevanta factori: ", criterii)
plot_varianta(alpha, criterii, 70, eticheta_x='Factor')

nr_factori = int(np.nanmean(criterii))

# Analiza corelatii variabile-factori
l = model_af.loadings_
t_l = pd.DataFrame(l, variabile_observate, etichete_factori)
t_l.to_csv('Corelatii_variabile-factori.csv')
corelograma(t_l)
for k in range(1, nr_factori):
    scatter(t_l, "F1", "F" + str(k + 1), "Plot corelatii factoriale",
True)

# Calcul scoruri
f = model_af.transform(x)
t_f = pd.DataFrame(f, date.index, etichete_factori)
t_f.to_csv('scor_f.csv')
for k in range(1, nr_factori):
    scatter(t_f, 'F1', 'F' + str(k + 1))

# Calcul comunalitati si varianta specificata
h = model_af.get_communalities()
psi = model_af.get_uniquenesses()
t_varianta_extrasa = pd.DataFrame({
    "Comunalitati": h,
    "Varianta specifica": psi
}, variabile_observate)
t_varianta_extrasa.to_csv("Varianta_extrasa.csv")
corelograma(t_varianta_extrasa, 0, "Varianta extrasa", "Reds")

show()
```

AD – discriminanta

1. FUNCTII

```
import numpy as np
import pandas as pd
from sklearn.metrics import confusion_matrix, cohen_kappa_score


def evaluare_model(y, predictie, clase):
    c = confusion_matrix(y, predictie)
    acuratete = np.diag(c) * 100 / np.sum(c, axis=1)
    acuratete_medie = np.mean(acuratete)
    acuratete_globala = sum(np.diag(c)) * 100 / len(y)
    kappa = cohen_kappa_score(y, predictie)
    t_conf = pd.DataFrame(c, index=clase, columns=clase)
    t_conf['Acuratete'] = acuratete
    t_acuratete = pd.Series([acuratete_globala, acuratete_medie, kappa],
                            ["Acuratete globala", "Acuratete medie",
"Index Cohen-Kappa"],
                            name="Acuratete")
    return t_acuratete, t_conf
```

2. GRAFICE

```
import matplotlib.pyplot as plt
from seaborn import kdeplot, scatterplot


def plot_distributii(t, v, g):
    fig = plt.figure(figsize=(9, 6))
    ax = fig.add_subplot(1, 1, 1)
    ax.set_title('Distributie pentru variabila' + v)
    kdeplot(t, x=v, hue=g, ax=ax, fill=True)
    plt.savefig('Distributie' + v)


def plot_scoruri(t, v1, v2, g):
    fig = plt.figure(figsize=(9, 6))
    ax = fig.add_subplot(1, 1, 1, aspect=1)
    ax.set_title('Plot scoruri discriminante')
    scatterplot(t, x=v1, y=v2, hue=g, ax=ax)
    plt.savefig('plot' + v1 + "_" + v2)


def show():
    plt.show()
```

3. MAIN

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from scipy.stats import f
from grafice import *
from functii import evaluare_model
from sklearn.naive_bayes import GaussianNB

date_antrenare_testare = pd.read_csv("park.csv", index_col=0)

variabile = list(date_antrenare_testare)
predictori = variabile[:-1]
clasa = variabile[-1]

# Splitare
x_train, x_test, y_train, y_test = train_test_split(
    date_antrenare_testare[predictori],
    date_antrenare_testare[clasa],
    test_size=0.4
)

# Modelul liniar
m_lin = LinearDiscriminantAnalysis()
m_lin.fit(x_train, y_train)

# Calculul puterii de discriminare a variabilelor predictor
# Validarea predictorilor
matrice_cov = np.cov(x_train.values, rowvar=False)  # T
medie_variab_antrenare = np.mean(x_train.values, axis=0)  # X_
medii_clase = m_lin.means_  # G
matrice_diaglonala = np.diag(m_lin.priors_)  # DG
```

```
masura_variatiei_intre_clase = (medii_clase - medie_variab_antrenare).T
@ matrice_diaglonala \
                                @ (medii_clase - medie_variab_antrenare)
# B
variatia_in_clase = matrice_cov - masura_variatiei_intre_clase  # W
clase = m_lin.classes_
q = len(clase)  # nr de clase
n = len(x_train)  # nr total de obs de antrenare
putere_discriminare = (np.diag(masura_variatiei_intre_clase) / (q - 1))
\
                      / (np.diag(variatia_in_clase) / (n - q))
p_value = 1 - f.cdf(putere_discriminare, q - 1, n - q)
# f.cdf(x, dfn, dfd) e functia de distributie cumulata
# x e val pentru care se calculeaza
# dfn nr de grade de lib pt numarator
# dfd -"- pentru numitor
t_pred = pd.DataFrame({
    "Putere discriminare": putere_discriminare,
    "P_Values": p_value
}, predictori)
t_pred.to_csv('Predictori.csv')

# Analiza discriminatori pe setul de antrenare
m = len(predictori)
nr_disc = min(m, q - 1)
z = m_lin.transform(x_train)
etichete_z = ["z" + str(i) for i in range(1, nr_disc + 1)]

# Calcul putere discriminare discriminatori pe setul de antrenare
T_z = np.cov(z, rowvar=False)
Z_ = np.mean(z, axis=0)
G_z = m_lin.transform(medii_clase)
DG = np.diag(m_lin.priors_)
B_z = (G_z - Z_).T @ DG @ (G_z - Z_)
W_z = T_z - B_z
putere_disc = (np.diag(B_z) / (q - 1)) / (np.diag(W_z) / (n - q))
p_value_z = 1 - f.cdf(putere_disc, q - 1, n - q)
t_discrim = pd.DataFrame({
    "Putere discriminare": putere_disc,
    "P_Values": p_value_z
}, etichete_z)
t_discrim.to_csv("Discriminatori.csv")

# Grafice distrib pt discriminatori pe setul de antrenare
tz = pd.DataFrame(z, x_train.index, etichete_z)
tz.to_csv("z.csv")
for v in etichete_z:
    plot_distributii(tz, v, y_train)
# Plot scoruri discriminate pe setul de antrenare
if nr_disc > 1:
    for i in range(2, q):
        plot_scoruri(tz, 'z1', 'z', str(i), y_train)

# Clasificare prin modelul liniar
# Testare model
predictie_test_lin = m_lin.predict(x_test)
t_acuratete_lin, t_conf_lin = evaluare_model(y_test, predictie_test_lin,
clase)
t_acuratete_lin.to_csv("Acuratete_lin.csv")
t_conf_lin.to_csv('MatConf.csv')
```

```
# Tabel predictii test
t_pred_test = pd.DataFrame(
    {
        clasa: y_test
    }, x_test.index
)
t_pred_test["Predictie Liniara"] = predictie_test_lin

# Aplicare model
set_aplicare = pd.read_csv("park_apply.csv", index_col=0)

predictie_lin = m_lin.predict(set_aplicare[predictori])
set_aplicare['Predictie Liniara'] = predictie_lin

# Modelul bayesian
m_bayes = GaussianNB()
m_bayes.fit(x_train, y_train)

# Testare
predictie_test_bayes = m_bayes.predict(x_test)
t_acuratete_bayes, t_conf_bayes = evaluare_model(y_test,
predictie_test_bayes, clase)
t_acuratete_bayes.to_csv("Acuratete_bayes.csv")
t_conf_bayes.to_csv("MatConf_bayes.csv")

t_pred_test["Predictie Bayes"] = predictie_test_bayes

# Aplicare model
predictie_bayes = m_bayes.predict(set_aplicare[predictori])
set_aplicare["Predictie Bayes"] = predictie_bayes

set_aplicare.to_csv("Predictii.csv")

# Analize erori
t_erori_lin = t_pred_test[predictie_test_lin != y_test]
t_erori_bayes= t_pred_test[predictie_test_bayes != y_test]
t_erori_lin.to_csv('Erori_lin.csv')
t_erori_bayes.to_csv('Erori_bayes.csv')

show()
```

CLUSTERI

1. FUNCTII

```
import pandas as pd
from pandas.api.types import is_numeric_dtype


def nan_replace(t):
    assert isinstance(t, pd.DataFrame)
    for v in t.columns:
        if t[v].isna().any():
            if is_numeric_dtype(t[v]):
                t[v].fillna(t[v].mean(), inplace=True)
            else:
                t[v].fillna(t[v].mode()[0], inplace=True)
```

2. GRAFICE

```
import matplotlib.pyplot as plt
import numpy as np
from matplotlib.colors import LinearSegmentedColormap

from scikitplot.metrics import plot_silhouette


def plot_indecsi_silhouette(x, partitia, culori, metoda):
    fig = plt.figure(figsize=(10, 7))
    ax = fig.add_subplot(1, 1, 1)
    cmap = LinearSegmentedColormap.from_list("cmap", culori,
len(culori))
    plot_silhouette(x, partitia, "Plot scoruri Silhouette. Metoda:" +
metoda, ax=ax, cmap=cmap)
    plt.savefig("Silhouette_P_" + str(len(culori)))


def show():
    plt.show()


def scatter(t, var1, var2, g, culori, titlu="Plot partitie"):
    fig = plt.figure(titlu, figsize=(13, 7))
    assert isinstance(fig, plt.Figure)
    ax = fig.add_subplot(1, 1, 1, aspect=1)
    assert isinstance(ax, plt.Axes)
    ax.set_title(titlu)
    if g is not None:
        clase = np.unique(g)
        q = len(clase)
        for i in range(q):
            t_ = t[g == clase[i]]
            ax.scatter(t_[var1], t_[var2], color=culori[i],
label=clase[i])
        tg = t.groupby(by=g).mean()
        ax.scatter(tg[var1], tg[var2], c=culori, marker="s", s=300,
alpha=0.5)
        if len(t) < 50:
            for i in range(len(t)):
                ax.text(t[var1].iloc[i], t[var2].iloc[i], t.index[i])
    ax.legend()
    plt.savefig("Plot_P_" + str(len(culori)))


def histograme(t, variabila, partitie, culori):
    fig = plt.figure(figsize=(9, 7))
    assert isinstance(fig, plt.Figure)
    fig.suptitle("Histograme pentru variabila " + variabila)
    clase = np.unique(partitie)
    q = len(clase)
    min_max = (t[variabila].min(), t[variabila].max())
    ax = fig.subplots(1, q, sharey=True)
    for i in range(q):
        axe = ax[i]
        assert isinstance(axe, plt.Axes)
        axe.set_xlabel(str(clase[i]))
        axe.hist(t[partitie == clase[i]][variabila], range=min_max,
color=culori[i], rwidth=0.9)
    plt.savefig("Hist_P_" + str(q) + "_" + variabila)
```

```
def elbow(d, plot=True):
    m = len(d)
    # diferentele dintre distante
    y = d[1:] - d[:m - 1]
    # print("y:",y,sep="\n")
    k_max = np.argmax(y)
    k = m - k_max
    if plot:
        fig = plt.figure(figsize=(9, 7))
        ax = fig.add_subplot(1, 1, 1)
        # ax.scatter(k + 1, y[k + 1], c="r")
        assert isinstance(ax, plt.Axes)
        ax.set_title("Elbow")
        ax.set_xlabel("Jonctiuni", color="m")
        ax.set_ylabel("Distante", color="m")
        ax.plot(np.arange(1, m + 1), d)
        ax.scatter(k_max + 1, d[k_max], alpha=0.5, s=100, c="g",
label="Partitia cu " + str(k) + " clusteri")
        ax.axhline(d[k_max], c="c")
        ax.axhline(d[k_max + 1], c="c")
        ax.legend()
        plt.savefig("Elbow_" + str(k))
    return k_max, k
```

3. HCLUST

```
import numpy as np
from scipy.cluster.hierarchy import linkage, dendrogram
from pandas import Categorical
import matplotlib.pyplot as plt
from sklearn.metrics import silhouette_samples
from grafice import elbow


class hclust():
    def __init__(self, t, variabile, metoda="ward"):
        self.metoda = metoda
        self.x = t[variabile].values
        self.h = linkage(self.x, method=metoda)
        self.etichete = t.index
        self.calcul_partitie()

    def calcul_partitie(self, k=None):
        nr_jonctiuni = self.h.shape[0]
        n = nr_jonctiuni + 1
        if k is None:
            k_max, k = elbow(self.h[:, 2])
        else:
            k_max = nr_jonctiuni - k
        self.k = k
        self.threshold = (self.h[k_max, 2] + self.h[k_max + 1, 2]) / 2

    def plot_ierarhie(self, titlu="Plot Ierarhie"):
        fig = plt.figure(figsize=(12, 7))
        ax = fig.add_subplot(1, 1, 1)
        ax.set_title(titlu + ". Metoda:" + self.metoda)
        r_dendr = dendrogram(self.h, labels=self.etichete,
color_threshold=self.threshold, ax=ax)
```

```
        self.partitie(r_dendr)
        plt.show()

    def partitie(self, r_dendr):
        culori_singleton = r_dendr["leaves_color_list"]
        self.culori_etichete = np.empty(len(self.etichete),
dtype=object)
        self.culori_etichete[np.array(r_dendr["leaves"])] =
np.array(culori_singleton)
        self.culori = list(np.unique(self.culori_etichete))
        self.k = len(self.culori)
        coduri = Categorical(self.culori_etichete).codes
        self.p = np.array(["c" + str(cod + 1) for cod in coduri])
        self.s_index = silhouette_samples(self.x, self.p)
        plt.savefig("dendrograma_" + str(self.k))
```

4. MAIN

```
from hclust import hclust
import pandas as pd
from grafice import *
from sklearn.decomposition import PCA
from functii import nan_replace
from scipy.cluster.hierarchy import linkage, dendrogram
from sklearn.metrics import silhouette_samples, silhouette_score

date = pd.read_csv('ADN_Tari.csv', index_col=0)
variabile = list(date)[1:]

valori_lipsa = date.isna().any().any()
if valori_lipsa:
    nan_replace(date)

metoda = 'ward'

m_hclust = hclust(date, variabile, metoda=metoda)
# trasare dendograma
m_hclust.plot_ierarhie()

partitie = m_hclust.p
culori = m_hclust.culori
s_index = m_hclust.s_index
plot_indecsi_silhouette(m_hclust.x, partitie, culori, metoda)
tabel_partitie = pd.DataFrame(
    {
        "Partitie_" + str(m_hclust.k): partitie,
        "Scor Silhouette": s_index
    }, date.index
)
tabel_partitie.to_csv("Partitie_" + str(m_hclust.k) + ".csv")

# Calcul axe principale pentru trasare plot partitie
pca = PCA(2)
pca.fit(m_hclust.x)
tz = pd.DataFrame(pca.transform(m_hclust.x), date.index, ['z1', 'z2'])
scatter(tz, 'z1', 'z2', partitie, culori)

for variabila in variabile:
    histograme(date, variabila, partitie, culori)
```

```
show()

# Calcul si analiza partitie cu k clusteri
# Exemplu pe partitia de 5 clusteri

k = 5
m_hclust.calcul_partitie(k)
# Calcul Silhouette
silhouette_k = silhouette_score(m_hclust.x, m_hclust.p)
index_silhouette_p_k = silhouette_samples(m_hclust.x, m_hclust.p)

# Afisare rezultate
print('Partitia din ', k, 'clusteri')
print('Scor silhouette', silhouette_k)
print('Partitia: ')
for v in zip(date.index, m_hclust.p, index_silhouette_p_k):
    print(v)

# Trasare plot Silhouette pentru partitia cu k clusteri
plot_indecsi_silhouette(m_hclust.x, m_hclust.p, m_hclust.culori, metoda)

# Trasare plot PCA pentru partitia cu k clusteri
pca = PCA(n_components=2)
pca.fit(m_hclust.x)
tz = pd.DataFrame(pca.transform(m_hclust.x), index=date.index,
columns=['z1', 'z2'])
scatter(tz, 'z1', 'z2', m_hclust.p, m_hclust.culori, titlu='Plot
partitie cu ' + str(k) + ' clusteri')

# Trasare dendrograma cu pragul pentru k clusteri
fig = plt.figure(figsize=(12, 7))
ax = fig.add_subplot(1, 1, 1)
ax.set_title('Plot ierarhie cu ' + str(k) + ' clusteri. Metoda: ' +
metoda)
dendrogram(m_hclust.h, labels=m_hclust.etichete,
color_threshold=m_hclust.threshold, ax=ax)
plt.axhline(y=m_hclust.threshold, color='black', linestyle='--')
plt.show()

# Trasare histograme pentru fiecare variabilă în funcție de clustere
for variabila in variabile:
    histograme(date, variabila, m_hclust.p, m_hclust.culori)

# Salvare partitie într-un fișier CSV
t_partitii = pd.DataFrame({'Partitie_' + str(k): m_hclust.p, 'Scor
Silhouette P_' + str(k): index_silhouette_p_k},
                          index=date.index)
t_partitii.to_csv('Partitie_' + str(k) + '.csv')

# Afișare finală a tuturor vizualizărilor
show()
```

CANONICA

1. FUNCTII

```
import numpy as np
import pandas as pd
from sklearn.cross_decomposition import CCA
from sklearn.preprocessing import normalize
from scipy.stats import chi2
import matplotlib.pyplot as plt
from seaborn import heatmap
from pandas.api.types import is_numeric_dtype


def nan_replace(t):
    assert isinstance(t, pd.DataFrame)
    for v in t.columns:
        if any(t[v].isna()):
            if is_numeric_dtype(t[v]):
                t[v].fillna(t[v].mean(), inplace=True)
            else:
                t[v].fillna(t[v].mode()[0], inplace=True)


# Funcție pentru testul Bartlett de semnificație a rădăcinilor canonice.
# r2: Coeficienții de determinare ai modelelor de regresie.
# n: Numărul de observații.
# p: Numărul de variabile independente.
# q: Numărul de variabile dependente.
# m: Numărul de rădăcini testate.
# Returnează p-values pentru testul Bartlett.
def test_bartlett(r2, n, p, q, m):
    x = 1 - r2
    df = [(p - k + 1) * (q - k + 1) for k in range(1, m + 1)]  # Grade de
libertate
    l = np.flip(np.cumprod(np.flip(x)))  # Inversarea ordinii elementelor
    chi2_ = (-n + 1 + (p + q + 1) / 2) * np.log(l)
    return 1 - chi2.cdf(chi2_, df)


# t_rxz: DataFrame pentru primul set de date (z1, z2).
# z1, z2: Numele coloanelor pentru variabilele pe axe din t_rxz.
# t_ryu: DataFrame pentru al doilea set de date (u1, u2).
# u1, u2: Numele coloanelor pentru variabilele pe axe din t_ryu.
def plot_corelatii(t_rxz, z1, z2, t_ryu, u1, u2):
    fig = plt.figure(figsize=(7, 7))
    ax = fig.add_subplot(1, 1, 1, aspect=1)
    ax.set_title('Plot corelatii')
    ax.set_xlabel(z1 + '/' + u1)
    ax.set_ylabel(z2 + '/' + u2)
    alpha = np.arange(0, np.pi * 2, 0.01)
    ax.plot(np.cos(alpha), np.sin(alpha))
    ax.axvline(0)
    ax.axhline(0)
    ax.scatter(t_rxz[z1], t_rxz[z2], c='r', label='X')
    ax.scatter(t_ryu[u1], t_ryu[u2], c='b', label='Y')
    p = len(t_rxz)
    for i in range(p):
        ax.text(t_rxz[z1][i], t_rxz[z2][i], t_rxz.index[i])
    q = len(t_ryu)
```

```
    for i in range(q):
        ax.text(t_ryu[u1][i], t_ryu[u2][i], t_ryu.index[i])
    ax.legend()


# t: Matricea de corelații.
# vmin, vmax: Valorile minime și maxime pentru scala de culori.
# titlu: Titlul figurii.
# cmap: Harta de culori pentru corelogramă.
def corelograma(t, vmin=-1, vmax=1, titlu="", cmap='coolwarm'):
    fig = plt.figure(figsize=(10, 7))
    ax = fig.add_subplot(1, 1, 1)
    ax.set_title(titlu)
    heatmap(t, vmin=vmin, vmax=vmax, cmap=cmap, annot=True, ax=ax)


def tabelare_matrice(x, nume_linii=None, nume_coloane=None, out=None):
    t = pd.DataFrame(x, nume_linii, nume_coloane)
    if out is not None:
        t.to_csv(out)
    return t


# z: Scorurile canonice pentru primul set de variabile.
# u: Scorurile canonice pentru al doilea set de variabile.
# k1, k2: Indicii componentelor canonice pe care să le traseze.
# nume_instante: Lista cu numele instanțelor.
def biplot(z, u, k1=0, k2=1, nume_instante=None):
    fig = plt.figure(figsize=(7, 7))
    ax = fig.add_subplot(1, 1, 1)
    ax.set_title("Plot instanțe", color="b", fontsize=16)
    ax.set_xlabel("z" + str(k1 + 1) + "/u" + str(k1 + 1))
    ax.set_ylabel("z" + str(k2 + 1) + "/u" + str(k2 + 1))
    ax.scatter(z[:, k1], z[:, k2], c="r", label="X")
    ax.scatter(u[:, k1], u[:, k2], c="b", label="Y")
    if nume_instante is not None:
        n = len(nume_instante)
        for i in range(n):
            ax.text(z[i, k1], z[i, k2], nume_instante[i])
            ax.text(u[i, k1], u[i, k2], nume_instante[i])
    ax.legend()


def show():
    plt.show()
```

2. MAIN

```
import numpy as np
import pandas as pd
from functii import *
from scipy.stats import chi2
from sklearn.cross_decomposition import CCA
from sklearn.preprocessing import normalize


# Citire date
t1 = pd.read_csv('mortalitate.csv', index_col=1)
variab1 = list(t1)[1:]
```

```
t2 = pd.read_csv('teritorial.csv', index_col=1)
variab2 = list(t2)[3:]

# Înlocuire valori lipsă
nan_replace(t1)
nan_replace(t2)

# Unire date
t = t1[variab1].join(t2[variab2], how='inner')
nume_instante = t.index

# Selectare variabile X și Y
x = t[variab1].values
y = t[variab2].values

n, p = x.shape  # n - nr de instante (randuri)
_, q = y.shape

m = min(p, q)  # m- nr de radacini posibile (coloane)

et_z = ["z" + str(i + 1) for i in range(m)]
et_u = ["u" + str(i + 1) for i in range(m)]
et_rad = ["rad" + str(i + 1) for i in range(m)]

# Construire model CCA
model_cca = CCA(m)
model_cca.fit(x, y)

# Calcul scoruri canonice (variabile canonice) z,u
z, u = model_cca.transform(x, y)

z_std = np.std(z, axis=0, ddof=1)
u_std = np.std(u, axis=0, ddof=1)

# Normalizare variabile canonice
normalize(z, axis=0, copy=False)
normalize(u, axis=0, copy=False)

# Identificare radacini semnificative (zk, uk)
lista_r = list()
for i in range(m):
    lista_r.append(np.corrcoef(z[:, i], u[:, i])[0, 1])

array = np.array(lista_r)
r2 = array * array

# Test Bartlett pentru semnificația rădăcinilor canonice
p_values = test_bartlett(r2, n, p, q, m)
print('p_values test_bartlett', p_values)

tabel_radacini = pd.DataFrame(
    data={
        'R': np.round(array, 3),
        'R2': np.round(r2, 3),
        'p-values': np.round(p_values, 5)
    }, index=et_rad
)

tabel_radacini.to_csv('Radacini.csv')

criteriu_radacini = np.where(p_values > 0.05)
```

```
print('criteriu alegere: ', criteriu_radacini)

nr_rad_semnificative = criteriu_radacini[0][0]
print('nr rad semnificative ', nr_rad_semnificative)

# Calcul corelații dintre variabile observate și variabile canonice
r_xz = model_cca.x_loadings_ * z_std
tabel_r_xz = tabelare_matrice(r_xz, variab1, et_z, "r_xz.csv")

r_yu = model_cca.y_loadings_ * u_std
tabel_r_yu = tabelare_matrice(r_yu, variab2, et_u, "r_yu.csv")

# Afișare grafice
for i in range(nr_rad_semnificative):
    plot_corelatii(tabel_r_xz, "z1", et_z[i], tabel_r_yu, "u1", et_u[i])
    show()

tabel_z = tabelare_matrice(z, nume_instante, et_z, "z.csv")
tabel_u = tabelare_matrice(u, nume_instante, et_u, "u.csv")

for i in range(nr_rad_semnificative):
    biplot(z, u, 0, i, nume_instante)
    show()

# Calcul varianță explicată și redundanță informațională
var_explicata = r2  # r2 a fost calculat anterior
red_inform = np.sum(r2)

print('Varianță explicată:', var_explicata)
print('Redundanță informațională:', red_inform)
```